

AI Agents and Agentic AI with
Python & Generative AI

Module 1

Agentic AI Concepts

- ✔ Introduction
Video • 10 min
- ✔ Flipped Interaction Pattern
Video • 7 min
- ✔ The Agent Loop
Video • 8 min
- ✔ Knowledge Check: Agent Loop & Flipped Interaction
Practice Assignment • Grade: 100%
- ✔ Running the Code Samples in the Course
Reading • 10 min
- ✔ Programmatic Prompting for Agents
Ungraded Plugin • 15 min
- ✔ Try Out Programmatic Prompting
Reading • 10 min
- ✔ Programmatic Prompting for Agents II
Ungraded Plugin • 15 min
- ✔ Programmatic Prompting for Agents III
Ungraded Plugin • 15 min
- ✔ Programmatic Prompting: Building Blocks for AI Agents
Dialogue • 15 min
- ✔ Try Out the Customer Service Agent
Reading • 10 min
- ✔ Giving Agents Memory
Ungraded Plugin • 15 min
- ✔ Practicing Programmatic Prompting for Agents
Ungraded Plugin • 15 min
- ✔ Practicing Programmatic Prompting for Agents (Solution)
Ungraded Plugin • 15 min
- ✔ Adding Structure to AI Agent Outputs
Video • 14 min
- ✔ Building Your First Agent
Ungraded Plugin • 15 min
- ✔ AI Agent / Environment Interface
Ungraded Plugin • 15 min
- ✔ AI Agent Feedback and Memory
Ungraded Plugin • 15 min
- ✔ The Agent Loop: From Understanding to Thinking Like an Agent
Dialogue • 15 min
- ✔ Learning More & Staying Connected
Reading • 10 min
- ✔ Understanding Agentic AI Concepts
Graded Assignment • Grade: 100%

Module 2

AI Agents, Tools, Actions, & Language

- GAU - Goals, Actions, Information, Language
Video • 7 min
- Giving Agents Tools
Video • 7 min
- Tool Descriptions and Naming
Video • 9 min
- Tool Results and Agent Feedback
Video • 10 min
- Agent Tools in Python**
Ungraded Plugin • 15 min
- Try Out an Agent that Calls Python Functions
Reading • 10 min
- Using Function Calling Capabilities with LLMs
Ungraded Plugin • 15 min
- Try Out LLM Function Calling
Reading • 10 min
- An Agent Loop with Function Calling
Ungraded Plugin • 15 min
- Try Out an Agent Loop with Function Calling
Reading • 10 min
- Exercise: Extend the Function Calling Agent
Reading • 10 min
- Agent Tool Design Best Practices
Ungraded Plugin • 15 min
- Understanding the AI Agent Loop
Graded Assignment • 30 min

Module 3

GAME: A Conceptual Framework for AI Agents

- Overview of the GAME Framework
Video • 5 min
- Designing AI Agents with GAME
Ungraded Plugin • 15 min
- Simulating Agents in ChatGPT
Video • 7 min
- Simulating GAME Agents in Conversation
Ungraded Plugin • 30 min
- Modular AI Agent Design
Ungraded Plugin • 15 min
- Agent Loop Customization
Ungraded Plugin • 15 min
- Implementing GAME in Code
Ungraded Plugin • 15 min
- Try Out the Agent Framework
Reading • 10 min
- How Your Agent Communicates with the LLM:
The Agent Language
Ungraded Plugin • 15 min
- Putting it All Together: Document Your Code
with a README Agent
Ungraded Plugin • 15 min

Module 4

Agent Tool Mangement

- Keeping Agent Tools Up to Date with Python Decorators
Ungraded Plugin • 15 min
- Tool Organization for Agents
Ungraded Plugin • 15 min
- Refactoring Our README Agent
Ungraded Plugin • 15 min
- Try Out the README Agent with the Decorator Reading • 10 min

Module 5

Rethinking How Software is Built in the Age of AI Agents

- Build the Impossible with AI Agents
Video • 1 min
- Rethinking How We Teach Innovation
Video • 4 min
- The Inventory Management Agent

<https://www.coursera.org/learn/ai-agents-python/ungradedWidget/RNyes/agent-tools-in-python>

Agent Tools in Python

AI-Agent Tool Descriptions and Naming

Describing Tools to the Agent

When developing an agentic AI system, one of the most critical aspects is ensuring that the agent understands the tools it has access to. In our previous tutorial, we explored how an AI agent interacts with an environment. Now, we extend that discussion to focus on tool definition, particularly the importance of naming, parameters, and structured metadata.

Example: Automating Documentation for Python Code

Imagine we are building an AI agent that scans through all Python files in a `src/` directory and automatically generates corresponding documentation files in a `docs/` directory. This agent will need to:

- **List Python files** in the `src/` directory.
- **Read the content** of each Python file.
- **Write documentation** files in the `docs/` directory.

Since file operations are straightforward for humans but ambiguous for an AI without context, we must clearly define these tools so the agent knows how to use them effectively.

Step 1: Defining a Tool with Structured Metadata

A basic tool definition in Python might look like this:

```
def list_python_files():
    """Returns a list of all Python files in the src/ directory."""
    return [f for f in os.listdir("src") if f.endswith(".py")]
```

This provides a function that retrieves all Python files in the `src/` directory, but for an AI system, we need a more structured way to describe it.

Step 2: Using JSON Schema to Define Parameters

When developers design APIs, they use structured documentation to describe available functions, their inputs, and their outputs. JSON Schema is a well-known format for defining APIs, making it a natural choice for AI agents as well.

For example, a tool that reads a file should specify that it expects a `file_path` parameter of type `string`. JSON Schema allows us to express this in a standardized way:

```
{
  "tool_name": "read_file",
  "description": "Reads the content of a specified file.",
  "parameters": {
    "type": "object",
    "properties": {
      "file_path": { "type": "string" }
    },
    "required": ["file_path"]
  }
}
```

Similarly, a tool for writing documentation should define that it requires a `file_name` and `content` :

```
{
  "tool_name": "write_doc_file",
  "description": "Writes a documentation file to the docs/ directory.",
  "parameters": {
    "type": "object",
    "properties": {
      "file_name": { "type": "string" },
      "content": { "type": "string" }
    },
    "required": ["file_name", "content"]
  }
}
```

By providing a JSON Schema for each tool:

1. The AI can Recognize the tool's purpose.
2. The AI / Environment interface can validate input parameters before execution.

It may look strange that multiple parameters to a function are represented as an object. When we are getting the agent to output a tool / action selection, we are going to want it to output something like this:

```
{
  "tool_name": "read_file",
  "args": {
    "file_path": "src/file.py"
  }
}
```

The schema describes the overall dictionary that will be used to capture the "args" to the function, so it is described as an object.

Expand

✓ Completed

[Go to next item](#)

 Like

 Dislike[Report an issue](#)

Go to next item →